

DiffSoup

คู่มืออธิบายโค้ดทีละ Section — สามเหลี่ยมที่ “หาอนุพันธ์ได้” ด้วยการสุ่ม (Stochastic Differentiable Triangle) บน CUDA

```
 $\alpha = \text{sigmoid}(d / \sigma) \rightarrow \text{sample} \sim \text{Bernoulli}(\alpha) \rightarrow \partial L / \partial V$ 
```

Möller-Trumbore

Stochastic Opacity

CUDA Kernel

atomicAdd

Backward Pass

Philox RNG

เขียนสำหรับ: นักศึกษาวิศวกรรมคอมพิวเตอร์ที่เพิ่งจบปี 2 — อ่านจากศูนย์ได้
อ้างอิงงาน DiffSoup (Tojo, Bickel, Umetani — CVPR 2026)

เอกสารนี้มีอะไรบ้าง

เราจะไล่จาก “ภาพใหญ่” (โปรเจกต์นี้ทำอะไร ทำไม) → “คณิตศาสตร์ที่ต้องรู้” → แล้วค่อยเปิดโค้ดทีละไฟล์ทีละ section ทุก section จะมี 3 ส่วน: **โค้ดทำอะไร**, **ทำไมต้องทำแบบนั้น**, และ **จุดที่มือใหม่มักงง**

00	ภาพรวม: DiffSoup คืออะไร และเรากำลังแก้ปัญหาอะไร	3
01	หัวใจของโจทย์: ทำไม “การสุม” ถึงทำให้หาอนุพันธ์ได้	4
02	คณิตศาสตร์ปูพื้น: ริงลี, barycentric, sigmoid, chain rule	5
03	Step 0 — CPU reference (NumPy) ทำไมต้องมีตัวเทียบก่อน	7
04	ปูพื้น CUDA: thread / block / grid map ริงลี 1 เส้น → 1 thread	9
05	Step 1 — Forward pass: Möller–Trumbore บน GPU	10
06	Step 2 — Opacity window: เปลี่ยนระยะขอบเป็นความน่าจะเป็น α	12
07	Step 3 — Stochastic decision: โยนเหรียญด้วย Philox RNG	14
08	Step 4–6 — Backward pass + atomicAdd + การตรวจ gradient/race	16
09	build_colab.py — ประกอบทุกอย่างเป็น notebook เดียว	19
10	สรุปทั้งระบบ + คำศัพท์ (glossary)	20

00 DiffSoup คืออะไร และเรากำลังแก้ปัญหาอะไร

ลองนึกภาพว่าเรามีสามเหลี่ยม 1 รูปลอยอยู่ในอวกาศ และมี “ภาพเป้าหมาย” ที่อยากให้สามเหลี่ยมนี้ไปอยู่ในตำแหน่งพอดี คำถามคือ: เราจะ “ขยับมุมสามเหลี่ยม (vertex) ที่ละนิด” ให้มันเข้าใกล้เป้าหมายโดยอัตโนมัติได้อย่างไร?

วิธีที่วงการ Deep Learning ใช้คือ **gradient descent** — คำแนะนำว่า “ถ้าขยับมุมไปทางไหน ผลลัพธ์จะดีขึ้น” (นั่นคือ gradient หรืออนุพันธ์ $\partial L / \partial V$) แล้วขยับตามนั้นซ้ำๆ แต่จะทำแบบนี้ได้ ระบบเรนเดอร์ภาพ (rendering) ต้อง “หาอนุพันธ์ได้” (differentiable) เสียก่อน

ปัญหาใหญ่: การเรนเดอร์ปกติ “หาอนุพันธ์ไม่ได้”

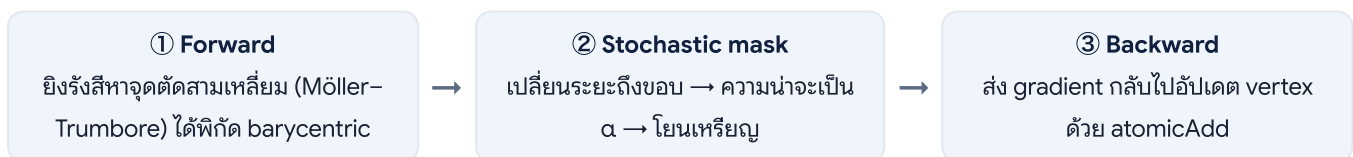
เวลาเช็คจุดหนึ่งอยู่ “ในสามเหลี่ยมหรือไม่” คำตอบเป็น **ใช่/ไม่ใช่** (0 หรือ 1) เป็นฟังก์ชันขั้นบันได ขยับมุมสามเหลี่ยมนิดเดียวค่าไม่เปลี่ยน (อนุพันธ์ = 0) จนกระทั่งขยับข้ามขอบพอดี ค่ากระโดดจาก 0 เป็น 1 ทันที (อนุพันธ์ = อนันต์) $\Rightarrow$ **gradient ใช้ไม่ได้ทั้งสองกรณี $\Rightarrow$ สอนให้มันเรียนรู้ไม่ได้**

โปรเจกต์นี้คือการ implement เทคนิคของงานวิจัย **DiffSoup** ที่แก้ปัญหานี้ด้วยไอเดียที่ฟังดูแปลกแต่ได้ผล: แทนที่จะตอบว่า “ทึบ/โปร่ง” แบบฟังก์ชัน ให้ตอบเป็น “ความน่าจะเป็นที่จะทึบ” แล้วโยนเหรียญตัดสิน การใส่ความสุ่มเข้าไปกลับทำให้ระบบหาอนุพันธ์ได้ (เดี่ยว Section 01 อธิบายว่าทำไม)

เป้าหมายของโค้ดชุดนี้ (test case)

- มีสามเหลี่ยม 1 รูป: $V_0(0,0,0)$, $V_1(1,0,0)$, $V_2(0,1,0)$ วางบนระนาบ $z = 0$
- ยิงรังสี (ray) จำนวน $N = 1,000,000$ เส้น เข้าหาสามเหลี่ยม โดยเจาะจงบริเวณ “ขอบ” เพราะ gradient ทั้งหมดอยู่ที่ขอบ
- มีพารามิเตอร์ $\sigma = 0.01$ คุมความ “คม/นุ่ม” ของขอบ
- ผลที่ต้องการ: คำนวณ gradient $\partial L / \partial V_0$, $\partial L / \partial V_1$, $\partial L / \partial V_2$ ได้ถูกต้อง บน GPU โดยไม่เกิด **race condition**

pipeline ทั้งหมด มองเป็น 3 ก้อน



หมายเหตุเรื่องทิศทาง (อ่านผ่านได้ ไม่กระทบความเข้าใจ)

งานวิจัยต้นฉบับทำ **rasterization** (ฉายสามเหลี่ยมลงจอภาพ) แต่โจทย์นี้ทำกลับด้านคือ **ray intersection** (ยิงรังสีเข้าหาสามเหลี่ยม) — มุมมองต่างกันแต่ใช้ trick หัวใจอันเดียวกันคือ stochastic opacity masking

01 ทำไม “การสุม” ถึงทำให้หาอนุพันธ์ได้

นี่คือ section ที่สำคัญที่สุดของทั้งเอกสาร ถ้าเข้าใจตรงนี้ที่เหลือคือรายละเอียด ขอบ่อยๆ ไล่

1) ปัญหา: ฟังก์ชันขั้นบันได

ให้ d = ระยะจากจุดที่รังสีตกถึงขอบสามเหลี่ยม (เป็นบวกเมื่ออยู่ข้างใน, เป็นลบเมื่ออยู่ข้างนอก) การตัดสินใจแบบเดิมคือ:

```
if d > 0: opaque = 1 # อยู่ในสามเหลี่ยม → ทึบ
else:     opaque = 0 # อยู่นอก → โปร่ง
```

กราฟของ `opaque` เทียบกับตำแหน่ง vertex เป็นเส้นแบนราบ แล้วกระโดด $\Rightarrow$ ความชัน (อนุพันธ์) เป็น 0 เกือบทุกที่ และพุ่งเป็นอนันต์ตรงรอยกระโดด **gradient descent** จึงไม่มี “ทิศทาง” ให้เดินตาม

2) วิธีแก้ขั้นแรก: ทำขอบให้ “นุ่ม” ด้วย sigmoid

แทนที่จะกระโดด $0 \rightarrow 1$ ทันที เราใช้ฟังก์ชัน **sigmoid** ทำให้เปลี่ยนแบบค่อยเป็นค่อยไป:

```
 $\alpha = \text{sigmoid}(d / \sigma) = 1 / (1 + e^{-(d/\sigma)})$ 
```

- อยู่ลึกในสามเหลี่ยม ($d \gg \sigma$): $\alpha \rightarrow 1$ (เกือบทึบแน่ๆ)
- อยู่บนขอบพอดี ($d = 0$): $\alpha = 0.5$
- อยู่นอกไกลๆ ($d \ll -\sigma$): $\alpha \rightarrow 0$ (เกือบโปร่งแน่ๆ)

ตอนนี้ α เป็นฟังก์ชันเรียบ (smooth) เทียบกับ d และ d ก็ขึ้นกับตำแหน่ง vertex $\Rightarrow$ **หาอนุพันธ์ $\partial\alpha/\partial v$ ได้แล้ว!** ค่า σ คือความกว้างของแถบ “นุ่ม” รอบขอบ: σ เล็ก = ขอบคม, gradient กระจุกในแถบแคบ

แต่... ภาพที่ render จริงต้องเป็น 0 หรือ 1 (พิกเซลทึบหรือโปร่ง) ไม่ใช่ 0.5

เราอยากได้ขอบคมในภาพจริง แต่ก็อยากได้ gradient เรียบไว้เรนเดอร์ — ขัดกันเอง ทางออกคือ “แยกร่าง”: ภาพจริงใช้การสุม (ได้ 0/1 คมๆ) ส่วน gradient ให้ไหลผ่านค่า α ที่เรียบ

3) ใส่การสุม: Bernoulli(α)

แต่ละรังสีสุ่มเลข ξ จาก 0 ถึง 1 อย่างสุ่มเท่าเทียม แล้วตัดสินใจ:

```
 $\xi \sim \text{Uniform}(0,1]$ 
sample = 1 (ทึบ)   if  $\xi < \alpha$ 
          0 (โปร่ง) otherwise
```

ผลลัพธ์แต่ละครั้งเป็น 0 หรือ 1 คมๆ (ดีต่อภาพ) แต่ **ค่าคาดหวัง** (ค่าเฉลี่ยถ้าทำซ้ำเยอะๆ) คือ:

```
 $E[\text{sample}] = P(\xi < \alpha) = \alpha$ 
```

กล่าวคือ ถ้ายิงรังสีหลายๆ เส้นที่จุดเดียวกัน แล้วเฉลี่ยผล 0/1 ที่ได้ มันจะ **ลู่เข้าหา α พอดี** (นี่คือหลัก Monte Carlo) และ α นี้แหละที่เรียบและหาอนุพันธ์ได้

กฎเจสำคัญ — “แยกเส้นทาง” ระหว่าง forward กับ backward

- **Forward (สร้างภาพ):** ใช้การสุ่ม $\rightarrow$ ได้ 0/1 คมๆ ดูเหมือนขอบจริง
 - **Backward (หา gradient):** ไม่หาอนุพันธ์ของการสุ่ม (ทำไม่ได้) แต่หาอนุพันธ์ของ **ค่าคาดหวัง** $E[\text{sample}] = \alpha(V)$ ซึ่งเรียบ
- $\Rightarrow \partial L / \partial V = \sum_i (\partial L / \partial \alpha_i) \cdot (\partial \alpha_i / \partial V)$ — นี่คือทั้งหมดที่ Step 4 ทำ

เทคนิคนี้เป็นญาติกับ **score-function estimator / REINFORCE** ใน reinforcement learning — หลักการเดียวกันคือ “เปลี่ยนปัญหา discrete ให้เป็นการหาอนุพันธ์ของค่าคาดหวัง” โค้ดชุดนี้เลือกเส้นทางที่ตรงกว่า (pathwise) คือหาอนุพันธ์ของ α ตรงๆ ซึ่ง **variance ต่ำกว่า**

ปัญหา	สาเหตุ	ทางแก้ในโค้ดนี้
Binary opacity หาอนุพันธ์ไม่ได้	ฟังก์ชันขั้นบันได $\partial/\partial V = 0$	$\alpha = \text{sigmoid}(d/\sigma) +$ สุ่ม Bernoulli
1 ล้าน thread เขียน gradient ของ vertex เดียวกัน	vertex 3 ตัวถูกใช้ร่วมโดยทุกรังสี	atomicAdd ลง buffer กลาง

02 คณิตศาสตร์พื้นฐาน (ฉบับพอใช้งาน)

ไม่ต้องเป็นเขียนเลข แค่รู้ชื่อเรื่องนี้ก็อ่านโค้ดรู้เรื่อง: รังสี, barycentric, Möller–Trumbore, และ chain rule

① รังสี (Ray)

รังสีคือจุดเริ่ม `origin` บวกทิศทาง `direction` คูณระยะ `t`:

$$P(t) = \text{origin} + t \cdot \text{direction} \quad (t > 0 \text{ คือไปข้างหน้า})$$

ในโค้ดนี้ รังสีส่วนใหญ่เริ่มเหนือระนาบ ($z=+1$) แล้วยิงลง ($\text{direction} = [0,0,-1]$) ไปตัดสามเหลี่ยมที่ $z=0$

② Barycentric coordinates (w, u, v)

จุดใดๆ บนระนาบสามเหลี่ยมเขียนเป็นส่วนผสมถ่วงน้ำหนักของ 3 มุมได้:

$$P = w \cdot V_0 + u \cdot V_1 + v \cdot V_2 \quad \text{โดย } w + u + v = 1, \quad w = 1 - u - v$$

- ถ้า $w, u, v \geq 0$ ทั้งหมด $\Rightarrow$ จุดอยู่ **ในสามเหลี่ยม**
- ถ้าตัวใดตัวหนึ่ง = 0 $\Rightarrow$ จุดอยู่บน **ขอบ** (เช่น $w=0$ คืออยู่บนขอบตรงข้าม V_0)
- $\min(w, u, v)$ = ระยะถึงขอบที่ใกล้สุด (ในหน่วย barycentric); centroid = $(1/3, 1/3, 1/3)$

เชื่อมกับโจทย

โจทยให้ยิงรังสีบริเวณที่ barycentric เข้าใกล้ 0 ก็เพราะนั่นคือ “ขอบ” — บริเวณเดียวที่ gradient ไม่เป็นศูนย์ ยิ่งตรงกลางสามเหลี่ยมไม่มีประโยชน์ต่อการเรียนรู้

③ Möller–Trumbore — สูตรหาจุดตัดรังสีกับสามเหลี่ยม

เป็นวิธีมาตรฐานที่หา t, u, v พร้อมกันในครั้งเดียวโดยไม่ต้องหา สมการระนาบก่อน หัวใจคือใช้ **cross product** และ **dot product**:

```
# e1, e2 = ขอบสองด้านจาก V0
e1 = V1 - V0 ; e2 = V2 - V0
pvec = direction × e2          # cross product
det = pvec · e1                # ถ้า det≈0 รังสีขนานระนาบ → ไม่ตัด
tvec = origin - V0
u = (tvec · pvec) / det
qvec = tvec × e1
v = (direction · qvec) / det
t = (e2 · qvec) / det
```

ไม่ต้องท่องสูตร แค่รู้ว่า: ใส่รังสี + 3 มุม $\rightarrow$ ได้ (t, u, v) ถ้า $u, v, w \geq 0$ และ $t > 0$ แปลว่าโดนสามเหลี่ยมจริง ตัวเลข `det` ที่เป็น 0 คือสัญญาณว่ารังสีขนานกับระนาบ (ไม่มีจุดตัดที่ชัดเจน) ต้องกันไว้ไม่ให้หารด้วยศูนย์

④ Sigmoid และอนุพันธ์ของมัน

$$\alpha = \text{sigmoid}(x) = 1 / (1 + e^{(-x)})$$

$$d\alpha/dx = \alpha \cdot (1 - \alpha)$$

สวยมาก: อนุพันธ์เขียนด้วยตัวมันเอง

ในโค้ดเราใช้ $x = d/\sigma$ ดังนั้นโดย chain rule: $d\alpha/dd = \alpha(1-\alpha)/\sigma$ — บรรทัดนี้จะโผล่ตรงๆ ใน Step 4

⑤ Chain rule — เส้นเลือดของ backward pass

เราอยากได้ $\partial L / \partial V$ แต่ V ส่งผลต่อ L ผ่านลูกโซ่ $V \rightarrow d \rightarrow \alpha \rightarrow L$ chain rule บอกว่าให้คุณอนุพันธ์ที่ละข้อต่อ:

$$\partial L / \partial V = (\partial L / \partial \alpha) \cdot (\partial \alpha / \partial d) \cdot (\partial d / \partial V)$$

└ จากภายนอก ─┘ └ $\alpha(1-\alpha)/\sigma$ ─┘ └ เรขาคณิตของขอบ ─┘

Step 4 ก็แค่คำนวณสามก้อนนี้แล้วคูณกัน นี่คือเหตุผลที่เราต้องเข้าใจว่า d มาจากเรขาคณิตอะไร — เพราะต้องหา $\partial d / \partial V$ เอง

สรุป Section 02

รังสีให้จุดตัด $\rightarrow$ Möller-Trumbore แปลงเป็น barycentric $\rightarrow$ barycentric แปลงเป็นระยะถึงขอบ $d \rightarrow$ sigmoid แปลง d เป็นความน่าจะเป็น $\alpha \rightarrow$ chain rule ส่ง gradient ย้อนกลับทั้งสาย

03 Step 0 — ตัวเทียบบน CPU (ground truth)

กฎเหล็กของงานนี้: “อย่าเขียน CUDA + gradient รวดเดียวจบ” เพราะ debug สองอย่างพร้อมกันคือฝันร้าย Step 0 จึงเขียนเวอร์ชัน CPU ด้วย NumPy ที่ **ตรวจด้วยมือได้** ไว้เป็น “เฉลย” ให้ CUDA เทียบทีหลัง

func ray_triangle_intersect(...)

เป็น Möller–Trumbore แบบ **vectorized** — คำนวณรังสีทั้ง N เส้นพร้อมกันด้วย NumPy (ไม่มี for-loop) ซึ่งเป็นการ “ซ่อม” สิ่งที่ GPU จะทำนานกันจริงๆ ใน Step 1

```
pvec = np.cross(directions, e2)      # (N,3) cross ทุกเส้นพร้อมกัน
det  = pvec @ e1                     # (N,) dot product
parallel = np.abs(det) < EPS         # รังสีขนานระนาบ กันหารศูนย์
inv_det  = np.where(parallel, 0.0, 1.0/...)
u = np.einsum("ij,ij->i", tvec, pvec) * inv_det
v = np.einsum("ij,ij->i", directions, qvec) * inv_det
t = (qvec @ e2) * inv_det
w = 1.0 - u - v
hit = (~parallel) & (u>=0) & (v>=0) & (w>=0) & (t>EPS)
```

เกร็ดสำหรับมือใหม่

@ คือ dot product, `np.einsum("ij,ij->i", a, b)` คือ dot product ของแต่ละแถว (รวม row-wise) เป็นทริก ทำ batch dot product เร็วๆ บน NumPy ส่วนรังสีที่ **miss** จะถูกใส่เป็น **NaN** เพื่อบอกว่า “ไม่มีค่า”

func edge_distance_bary(bary)

```
return np.min(bary, axis=-1)      # min(w,u,v) = ระยะถึงขอบใกล้สุด
```

ค่า 0 = อยู่บนขอบพอดี, $\frac{1}{3}$ = centroid ใช้วัดว่ารังสี “เฉียดขอบ” แค่ไหน

func run_tests() — 12 เคสที่ตรวจด้วยมือได้

หัวใจของ Step 0 คือชุดเทสที่เรา **รู้คำตอบล่วงหน้า** เช่น ยิงรังสีลงตรง centroid ต้องได้ $\text{bary} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$; ยิงตรงมุม V_1 ต้องได้ $(0, 1, 0)$; ยิงนอกสามเหลี่ยมต้อง miss เคสพวกนี้ทำให้มั่นใจว่าสูตรถูกก่อนเอาไปขึ้น GPU

เคส	คาดหวัง	ทดสอบอะไร
centroid hit	$\text{bary}=(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$	กลางสามเหลี่ยม
edge midpoint V_1V_2	$\text{bary}=(0, \frac{1}{2}, \frac{1}{2})$	จุดบนขอบ ($w=0$)
just outside edge	miss	เลยขอบไปนิดเดียว
parallel to plane	miss	กันเคส $\text{det}\approx 0$
ray points away	miss	กันเคส $t<0$

`func` `edge_biased_sample(n)` — สร้างรังสีเกาะขอบ

ฟังก์ชันนี้สุ่มจุดบนสามเหลี่ยมแบบ `uniform` ก่อน (ด้วยสูตร `r1=vrand`, `u=1-r1`, `v=r1(1-r2)` ซึ่งเป็นสูตรมาตรฐานสุ่มจุดในสามเหลี่ยมให้กระจายเท่ากัน) แล้ว “ดึง” แต่ละจุดเข้าหาขอบที่ใกล้สุด (คูณ coordinate ที่เล็กสุดด้วย 0.5) เพื่อให้รังสีกระจุกใกล้ขอบ — ตรงกับที่โจทย์ต้องการ เพราะ gradient อยู่ใกล้ขอบ

ทำไม Step 0 ถึงสำคัญทั้งที่ยังไม่แตะ GPU

ทุก Step ถัดไปจะ import ฟังก์ชันจากไฟล์นี้เป็น “เจลย” แล้วเทียบ `np.allclose(...)` ถ้า CUDA ให้ผลต่างจาก CPU เกิน tolerance = มี bug รู้กันที่ นี่คือการพัฒนารูปแบบ `incremental + verifiable`

04 ปูพื้น CUDA: thread / block / grid

ก่อนอ่านไฟล์ `.cu` ต้องเข้าใจโมเดลการทำงานขนานของ GPU เสียก่อน ไม่เช่นนั้นจะงงกับบรรทัดแรกของทุก kernel

GPU ทำงานยังไง (ฉบับ 1 นาที)

CPU มีไม่ก็ core แต่แต่ละ core เก่งมาก เหมาะกับงานเรียงลำดับ ส่วน GPU มี core เล็กๆ **หลายพัน** ตัว เหมาะกับงานที่ทำ “อย่างเดียวกันซ้ำๆ กับข้อมูลคนละชิ้น” — ซึ่งคือ pattern ของเรafort: รังสี 1 ล้านเส้น ทำ Möller–Trumbore เหมือนกันหมด ต่างกันแค่ข้อมูล

ลำดับชั้น: thread → block → grid

- **thread** = คนงาน 1 คน ทำงานของรังสี 1 เส้น
- **block** = กลุ่ม thread (ในโค้ดนี้ = 256 threads/block)
- **grid** = กลุ่มของ block ทั้งหมด

แต่ละ thread หา “เลขประจำตัว” (index) ของตัวเองเพื่อรู้ว่าต้องหยิบข้อมูลชิ้นไหน — บรรทัดนี้จะเห็นในทุก kernel:

```
const int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i >= N) return;  # กัน thread ส่วนเกินไม่ให้อ่านเลย array
```

`blockIdx.x` = ฉันอยู่ block ที่เท่าไร, `blockDim.x` = block หนึ่งมีกี่ thread (256), `threadIdx.x` = ฉันเป็น thread ที่เท่าไรใน block ⇒ คูณบวกกันได้ index รวมทั่วทั้ง grid

ฝั่ง host (CPU) เป็นคนสั่งว่าจะใช้กี่ block กี่ thread — โพลีในทุกไฟล์:

```
const int threads = 256;
const int blocks = (N + threads - 1) / threads;  # ปัดขึ้นให้ครอบคลุม N ครบ
kernel<<<blocks, threads>>>(...);  # ไวยากรณ์ <<< >>> เฉพาะ CUDA
```

สูตร `(N + threads - 1) / threads` คือการหาร **ปัดขึ้น** เพื่อให้มี thread ครอบคลุมทุกรังสี (จึงต้องมี `if (i >= N) return;` กัน thread เกิน)

ของแถมที่เห็นบ่อยในโค้ดนี้

สิ่งที่เห็น	แปลว่า
<code>__global__</code>	ฟังก์ชันนี้รันบน GPU แต่เรียกจาก CPU (คือ kernel)
<code>__device__</code>	ฟังก์ชันช่วย รันบน GPU เรียกจาก GPU ด้วยกัน
<code>__restrict__</code>	สัญญาว่า pointer ไม่ซ้ำทับกัน ช่วยให้ compiler optimize
<code>scalar_t</code>	type ปรอท (template) ใช้ได้ทั้ง float32/float64 เขียนครั้งเดียวใช้ได้สองแบบ
<code>torch.cuda.synchronize()</code>	CPU รอให้ GPU ทำงานเสร็จจริงก่อนวัดเวลา/อ่านผล

JIT compile — ไม่ต้องมี setup.py

ทุกไฟล์ `.cu` ถูกคอมไพล์ตอนรันด้วย `torch.utils.cpp_extension.load(...)` รอบแรกช้า (~30-60 วินาที) รอบถัดไป cache ไว้ ทำให้รันบน Colab/เครื่อง lab ได้สะดวก (macOS รันไม่ได้เพราะไม่มี NVIDIA GPU)

05 Step 1 — Forward pass บน GPU

เอา Möller–Trumbore จาก Step 0 มาแปลงเป็น CUDA kernel แบบ **1 รังสี / 1 thread** ยังไม่มีการสุม แต่ตอบว่า “โดนไหม + barycentric เท่าไร” แล้วเทียบกับ CPU ให้ตรง

.cu intersect_kernel — หัวใจ

โครงเหมือน Step 0 เป๊ะ ต่างที่เขียนแบบทาง component (px, py, pz...) เพราะ GPU ไม่มี np.cross ให้ใช้ ต้องเขียน cross product เอง:

```
const int i = blockIdx.x*blockDim.x + threadIdx.x;
if (i >= N) return;
// อ่านรังสีของ thread นี้ (แถวที่ i)
const scalar_t ox = origins[3*i+0], oy = origins[3*i+1], oz = origins[3*i+2];
...
// pvec = direction × e2 (เขียน cross เอง)
const scalar_t px = dy*e2z - dz*e2y;
const scalar_t py = dz*e2x - dx*e2z;
const scalar_t pz = dx*e2y - dy*e2x;
const scalar_t det = px*e1x + py*e1y + pz*e1z;
const bool parallel = (det > -EPS) && (det < EPS);
const scalar_t inv_det = parallel ? 0 : 1/det;
...
const bool h = (!parallel) && (u>=0) && (v>=0) && (w>=0) && (t>EPS);
hit[i] = h;
t_out[i] = h ? t : NaN; // miss → เก็บ NaN เหมือน CPU
bary[3*i+0] = h ? w : NaN; ...
```

ทำไมต้องตรงกับ CPU แบบ component-by-component

เพราะ Step 0 กำหนด “convention” ไว้ว่า bary = (w,u,v) เรียงแบบนี้, miss = NaN ถ้า GPU เรียงสลับหรือคีนค่าต่าง การเทียบ np.allclose ใน step1_forward.py จะ FAIL ทันทีกี่ — เป็นการบังคับให้สองเวอร์ชัน “พูดภาษาเดียวกัน”

.cu intersect_forward — ผัง host (สะพานเชื่อม PyTorch)

ฟังก์ชันนี้รันบน CPU ทำหน้าที่: (1) ตรวจสอบ input ด้วย TORCH_CHECK (เช็คว่าอยู่บน CUDA, ขนาด (N,3) ถูก), (2) แปะ 3 vertex เป็น buffer เดียว 9 ตัว, (3) คำนวณจำนวน block, (4) ยิง kernel ผ่าน AT_DISPATCH_FLOATING_TYPES (มาโครที่เลือก float32/64 อัตโนมัติ)

```
const int threads = 256;
const int blocks = (N + threads - 1) / threads;
AT_DISPATCH_FLOATING_TYPES(origins.scalar_type(), "intersect_kernel", [&]{
    intersect_kernel<scalar_t><<<blocks, threads>>>( ... );
});
```

ส่วน PYBIND11_MODULE ท้ายไฟล์คือการ “เปิดประตู” ให้ Python เรียก ext.intersect_forward(...) ได้

`.py` step1_forward.py — ตรวจสอบ

ทำสองการทดสอบ: **(a)** 12 เคสมีอ (ชุดเดียวกับ Step 0) และ **(b)** ยิงรังสีเกาะขอบ N เส้น แล้วเทียบ bary/t ของ GPU (float32) กับ CPU (float64) ว่าต่างกันไม่เกิน tolerance

```
hit_g, t_g, bary_g = cuda_intersect(ext, origins, dirs, V0,V1,V2)
bary_err = np.abs(bary_c[both_hit] - bary_g[both_hit])
bary_ok = bary_err.max() < atol_bary    # 1e-5
```

จุดที่มือใหม่มักงง: float32 vs float64

CPU ใช้ float64 (แม่นยำสูง) เป็นเจเลย, GPU ใช้ float32 (เร็วกว่า/มาตรฐาน graphics) ผลจึง **ต่างกันนิดหน่อยตามธรรมชาติ** เราจึงเทียบด้วย “ต่างไม่เกิน tolerance” ไม่ใช่ “เท่ากันเป๊ะ” — เป็นเรื่องปกติของการคำนวณ floating point

06 Step 2 — Opacity window: ระยะขอบ $\rightarrow \alpha$

เพิ่มขั้นตอน “แปลงระยะถึงขอบเป็นความน่าจะเป็นที่ $\alpha = \text{sigmoid}(d/\sigma)$ ” ยังไม่สั่ม คึน α ออกมาดูว่ามัน “นุ่ม” ตามทฤษฎีใหม่

จาก barycentric $\rightarrow$ ระยะจริง (world units)

barycentric เป็นสัดส่วน ไม่ใช่ระยะจริง เราต้องแปลงเป็นระยะตั้งฉากถึงขอบในหน่วยจริง โดยใช้ความสัมพันธ์เรขาคณิต: ระยะตั้งฉากจากจุดถึงขอบที่ตรงข้าม vertex $i = \text{bary}_i \times (\text{ความสูงจาก vertex } i)$ และความสูง = $|n| / L_i$ เมื่อ $|n| = 2 \cdot \text{พื้นที่}$ และ $L_i = \text{ความยาวขอบตรงข้าม}$:

```
const scalar_t two_area = sqrt(nx*nx + ny*ny + nz*nz); // |e1 x e2| = 2*Area
const scalar_t d0 = w * (two_area / L0); // ระยะถึงขอบตรงข้าม V0
const scalar_t d1 = u * (two_area / L1);
const scalar_t d2 = v * (two_area / L2);
scalar_t d = min(d0, d1, d2); // ขอบที่ใกล้สุด
alpha[i] = stable_sigmoid(d / sigma);
```

.cu stable_sigmoid — ทำไมต้อง “stable”

```
if (x >= 0) return 1/(1+exp(-x));
else      return exp(x)/(1+exp(x));
```

ถ้าเขียน $1/(1+\exp(-x))$ ตรงๆ เวลา x ติดลบมากๆ $\exp(-x)$ จะระเบิดเป็นเลขมหาศาล (overflow) เป็น inf การแยกสองกรณีตามเครื่องหมายของ x ทำให้ $\exp$ รับแต่ค่าติดลบเสมอ (ผลอยู่ใน 0-1) จึงไม่ overflow — เป็น trick มาตรฐานเวลา implement sigmoid เอง

จุดสำคัญที่ต่างจาก Step 1: ไม่ทิ้งรังสีที่ “พลาด” สามเหลี่ยม

Step 1 รังสีที่อยู่นอกสามเหลี่ยม = miss (NaN) แต่ Step 2 **คำนวณ α ให้ด้วย** トラバใดที่ยังตัดระนาบข้างหน้า (เงื่อนไขแค่ $t > \text{EPS}$ ไม่เช็ค $u,v,w \geq 0$) เพราะรังสีที่ **คร่อมขอบ** (อยู่นอกนิดเดียว, d ติดลบนิดเดียว, α ใกล้ 0.5) คือบริเวณที่ gradient อยู่! ถ้าทิ้งไปจะไม่เหลืออะไรให้เรียนรู้

.py การทดสอบ — sweep ข้ามขอบ

step2_opacity.py ยิงรังสีไล่ตั้งฉากข้ามขอบ (y จาก -5σ ถึง $+5\sigma$) แล้วตรวจว่า:

- α ที่ $y=0$ (บนขอบ) = 0.5 พอดี
- α ลึกข้างใน $\rightarrow \sim 1$, นอกไกล $\rightarrow \sim 0$ อย่างนุ่มนวล (monotonic)
- ความกว้างแถบ 10-90% = $\ln(81) \cdot \sigma \approx 4.39\sigma$ — สูตรนี้พิสูจน์ได้จาก sigmoid ตรงๆ เป็นการยืนยันว่า σ คมคมของขอบจริง

```
expected_width = math.log(81.0) * sigma # ≈ 0.0439 เมื่อ σ=0.01
```

ทำไมต้องเช็คความกว้าง 10–90%

มันพิสูจน์ว่าโค้ดไม่ได้แค่ “ดูนุ่มๆ” แต่นุ่มตาม σ ที่กำหนด **เชิงปริมาณ** — ถ้าความกว้างผิด แปลว่าการแปลง barycentric → ระยะจริง หรือการหาร σ มีบั๊ก

07 Step 3 — Stochastic decision (โยนเหรียญ)

เอา α จาก Step 2 มาแปลงเป็นการตัดสินใจจริง 0/1 ด้วยการสุ่ม Bernoulli(α) จุดที่ยากคือ “สุ่มเลขบน GPU 1 ล้าน thread อย่างไม่ซ้ำ และทำซ้ำได้”

.cu การตัดสินใจด้วยการสุ่ม

```
// Philox: seed ร่วม, sequence ต่อ thread = index รังสี i
curandStatePhilox4_32_10_t state;
curand_init(seed, (unsigned long long)i, 0ULL, &state);
const float xi = curand_uniform(&state); //  $\xi \in (0,1]$ 
const scalar_t s = (xi < a) ? 1 : 0; // opaque  $\Leftrightarrow \xi < \alpha$ 
sample[i] = valid ? s : 0;
```

ทำไมต้องใช้ Philox (counter-based RNG)?

RNG แบบเดิม (เช่น Mersenne Twister) มี “สถานะ” ที่ต้องเดินต่อทีละขั้นแบบ sequential ไม่เหมาะกับ GPU ที่ทำงาน Philox เป็น **counter-based**: ป้อน (seed, index) แล้วได้เลขสุ่มออกมาทันทีโดยไม่ต้องเดินสถานะ — เหมาะกับ 1 ล้าน thread เพราะ:

- seed เดียวกัน + index ต่างกัน $\Rightarrow$ stream อิสระต่อกัน (รังสีแต่ละเส้นได้เลขสุ่มคนละชุด ไม่ correlate)
- seed เดียวกัน $\Rightarrow$ ผลเหมือนเดิมทุกครั้ง (reproducible) สำคัญมากเวลา debug

ทำไมต้อง reproducible?

เวลา debug ถ้าผลสุ่มเปลี่ยนทุกครั้งที่รัน เราจะแยกไม่ออกว่า “ผลต่างเพราะ bug หรือเพราะสุ่มได้คนละชุด” การ fix seed ให้ผลเดิมเป๊ะ ทำให้ทดสอบได้ว่าโค้ด deterministic จริง (เห็นใน test ข้อ b: seed เดียวกันต้องได้ bit-identical)

.py การทดสอบ 3 ด้าน

เทส	ตรวจอะไร	ผลที่คาดหวัง
(a) convergence	เฉลี่ย sample ที่จุดเดียว M ครั้ง	ลู่เข้า α (ภายใน ~ 4 std-error ของ Monte Carlo)
(b) reproducibility	seed เดิม vs seed ใหม่	seed เดิม=เหมือนเป๊ะ; seed ใหม่=ต่าง $\sim 50\%$ แต่ค่าเฉลี่ยเท่าเดิม
(c) smoothness	sweep ข้ามขอบ 10^6 เส้น แบ่ง bin	ค่าเฉลี่ยแต่ละ bin = sigmoid(d/σ), RMSE < 0.02

```
# (a) แถบความคลาดเคลื่อน Monte Carlo: ยิ่งสุ่มเยอะยิ่งแคบ  $\sim 1/\sqrt{M}$ 
mc = 4.0 * math.sqrt(a*(1-a)/M)
ok = abs(mean - a) < mc
```

แนวคิด Monte Carlo error

การเฉลี่ยเหรียญ M เหรียญ คลาดเคลื่อนจากค่าจริง $\sim \sqrt{(\alpha(1-\alpha)/M)}$ ยิ่ง M มากยิ่งแม่นยำ (ลดลงเป็น $1/\sqrt{M}$) test จึงไม่ได้
เชื่อว่า “เท่า α เป๊ะ” แต่เชื่อว่า “อยู่ในแถบความคลาดเคลื่อนที่ทฤษฎีอนุญาต” — เป็นวิธีทดสอบโค้ดที่มีความสุ่มอย่างถูกต้อง

08 Step 4–6 — Backward + atomicAdd + การตรวจ

ของจริงอยู่ตรงนี้: คำนวณ gradient $\partial L / \partial V$ ส่งกลับไปอัปเดต vertex โดยที่ 1 ล้าน thread เขียนลง buffer เดียวกันโดยไม่ทะเลาะกัน (race condition)

chain rule ในโค้ด

จาก Section 02 เราต้องการ $\partial L / \partial V = (\partial L / \partial \alpha) \cdot (\partial \alpha / \partial d) \cdot (\partial d / \partial V)$ โค้ดคำนวณทีละก้อน:

```
const scalar_t a = stable_sigmoid(d / sigma);
const scalar_t dadd = a*(1-a)/sigma;           //  $\partial \alpha / \partial d$ 
const scalar_t factor = grad_alpha[i] * dadd;   //  $(\partial L / \partial \alpha) \cdot (\partial \alpha / \partial d) = \partial L / \partial d$ 
```

เหลือก้อนสุดท้าย $\partial d / \partial V$ ซึ่งเป็นเรขาคณิตของระยะจากจุดถึงเส้นขอบ

.cu sdist2d — ระยะจากจุดถึงเส้น (2D)

```
// ระยะมีเครื่องหมาย จากจุด P ถึงเส้น A→B
cr = (ax - px)*ey - (ay - py)*ex;   // 2D cross
return cr / L;                      // L = ความยาวขอบ
```

เพราะ vertex อยู่บนระนาบ $z=0$ และรั้งสี่ข้างตามแกน z ปัญหาจึงยุบเหลือ 2D ทำให้สูตร gradient ง่ายลงมาก จุดสำคัญ: ระยะถึงขอบขึ้นกับ **vertex 2 ตัวที่เป็นปลายขอบนั้นเท่านั้น** $\Rightarrow$ vertex ที่อยู่ตรงข้ามขอบได้ gradient = 0

.cu $\partial d / \partial V$ — อนุพันธ์ของระยะเทียบปลายขอบ

```
//  $d = \text{cross} / L \rightarrow$  ใช้ quotient rule กับทั้ง cross และ L
dd_ax = (ey + (ay-py))/L - cr*(-ex/L)/L2;
dd_ay = (- (ax-px) - ex)/L - cr*(-ey/L)/L2;
dd_bx = (- (ay-py))/L - cr*( ex/L)/L2;
dd_by = ( (ax-px))/L - cr*( ey/L)/L2;
```

ไม่ต้องจำสูตร แค่เข้าใจว่ามันคืออนุพันธ์ของ $d = \text{cross}(A,B,P)/L$ เทียบพิกัด x,y ของปลายขอบ A และ B (ใช้ quotient rule เพราะทั้งเศษ cross และส่วน L ขึ้นกับ A,B)

.cu atomicAdd — แก้ race condition

```
atomicAdd(&grad_verts[3*ia+0], factor * dd_ax);
atomicAdd(&grad_verts[3*ia+1], factor * dd_ay);
atomicAdd(&grad_verts[3*ib+0], factor * dd_bx);
atomicAdd(&grad_verts[3*ib+1], factor * dd_by);
```

Race condition คืออะไร และ atomicAdd แก้ยังไง

รังสีทั้ง 1 ล้านเส้นต้องบวก gradient ลง vertex เพียง 3 ตัว (6 ช่อง x,y) ถ้าเขียน `grad += ...` ตรงๆ สอง thread อาจอ่านค่าเดิมพร้อมกัน บวกของตัวเอง แล้วเขียนทับกัน $\Rightarrow$ ข้อมูลหาย (lost update) `atomicAdd` รับประกันว่า “อ่าน-บวก-เขียน” เป็นก่อนเดียวที่ไม่มีใครแทรก $\Rightarrow$ ผลรวมถูกต้องเสมอ

กับดักที่ต้องแยกให้ออก: FP ordering $\neq$ race

การบวก floating point **ไม่สลับที่** ($a+b+c \neq c+b+a$ เป๊ะ ในระดับ bit) เพราะ `atomicAdd` ทำงานตามลำดับที่ thread มาถึง ซึ่งสุ่มๆ รอบ $\Rightarrow$ ผลต่างกันนิดในระดับ bit สุดท้าย **นี้ไม่ใช่ bug** (ผลถูกเสมอ) แต่ทำให้ผลไม่ bit-exact reproducible — ต้องไม่สับสนกับ race จริง

`.py` Step 5 — gradient check ด้วย finite differences

วิธีพิสูจน์ว่า gradient ถูก: ขยับ vertex ทีละนิด $\pm \epsilon$ แล้วดูว่า L เปลี่ยนเท่าไร เทียบกับ gradient ที่ kernel คำนวณ (**central difference**):

```
fd[vi,c] = (S(V+ $\epsilon$ ) - S(V- $\epsilon$ )) / (2 $\epsilon$ )    # ค่าจริงเชิงตัวเลข
rel = |analytic - fd|.max() / scale           # ต้อง < 1e-4
```

ทำใน **float64** และใช้ $L = \sum \alpha$ (ไม่ใช่ค่าสุ่ม) เพื่อให้ **ไม่มี Monte Carlo noise** — gradient ไหลผ่าน α ที่เรียบ ไม่ใช่ผ่าน sample ที่สุ่ม จึงเทียบได้แม่นยำถึง $\sim 1e-5$ และตรวจด้วยว่า gradient แกน $z = 0$ (เพราะขยับในระนาบ จุดตกไม่เลื่อน)

`.py` Step 6 — race / determinism check

รัน backward (float32) ซ้ำ 50 รอบด้วย input เดียวกัน วัด:

- **run-to-run spread** (max-min ระหว่างรอบ) — ต้องเล็กระดับ FP ordering
- **bias เทียบ float64** — ค่าเฉลี่ยต้องตรงเฉลย

```
no_race = (rel_spread < 1e-3) and (rel_bias < 1e-3)
```

ตรรกะการตัดสินใจ

ถ้าผลแกว่ง **เล็กน้อย** และเฉลี่ยตรงเฉลย = ไม่มี race (แกว่งเพราะ FP ordering เฉยๆ) แต่ถ้าผลแกว่ง **ใหญ่/มั่ว** = มี race จริง (lost update) นี่คือเหตุผลที่ test แยกสองกรณีนี้อย่างระมัดระวัง

09 build_colab.py — ประกอบเป็น notebook เดียว

macOS รัน CUDA ไม่ได้ (ไม่มี NVIDIA GPU) ไฟล์นี้จึงรวมซอร์สทั้งหมดเป็น Jupyter notebook (`.ipynb`) ให้อัปขึ้น Google Colab ที่มี GPU ฟรี (T4)

มันทำงานยังไง

เป็นสคริปต์ Python ที่ **สร้างไฟล์ notebook** โดยอ่านซอร์สทุกไฟล์มาเป็น string แล้วประกอบเป็น cell:

```
def writefile_cell(filename, body):
    # ใช้ %%writefile ให้ Colab เขียนไฟล์ลง working dir
    return code(f"%%writefile {filename}\n{body}")
```

เมจิก `%%writefile` ของ Jupyter จะเขียนเนื้อหา cell ลงเป็นไฟล์จริงบน Colab จากนั้น cell ถัดไปที่ `!python step1_forward.py` เพื่อรันทดสอบ

ลำดับ cell ใน notebook

1. เช็ค environment (มี GPU ใหม่, ติดตั้ง `ninja` ที่ torch JIT ต้องใช้)
2. เขียนไฟล์ซอร์สทั้ง 9 ไฟล์ลง Colab ด้วย `%%writefile`
3. รัน Step 0 (CPU sanity) → Step 1 → 2 → 3 → 4–6 ตามลำดับ

ผลลัพธ์เขียนออกเป็น `step1_colab.ipynb` (JSON ตาม nbformat 4) ตั้ง `accelerator: "GPU"` ไว้พร้อม

ทำไมออกแบบเป็น notebook

เพราะผู้พัฒนาเครื่องเป็น macOS (รัน CUDA ไม่ได้) แต่ Colab มี GPU ฟรี การ generate notebook อัตโนมัติทำให้ไม่ต้อง copy-paste โค้ดทีละไฟล์ และมั่นใจว่า notebook ตรงกับซอร์สล่าสุดเสมอ (single source of truth)

ภาพรวมความสัมพันธ์ของไฟล์ทั้งหมด

ไฟล์	บทบาท	พึ่งพา
<code>step0_cpu_reference.py</code>	เฉลย CPU + ตัวสร้างรังสี	NumPy
<code>step1_cuda_forward.cu</code>	kernel หาจุดตัด	—
<code>step1_forward.py</code>	ทดสอบ Step 1 vs Step 0	step0
<code>step2_cuda_opacity.cu</code>	kernel $\alpha = \text{sigmoid}(d/\sigma)$	—
<code>step2_opacity.py</code>	ทดสอบความนุ่ม + σ	step0
<code>step3_cuda_stochastic.cu</code>	kernel สุ่ม Bernoulli+Philox	—
<code>step3_stochastic.py</code>	ทดสอบ converge/repro/smooth	step0

<code>step4_cuda_backward.cu</code>	kernel backward+atomicAdd	—
<code>step4_backward.py</code>	gradient check + race check	step0
<code>build_colab.py</code>	ประกอบทุกอย่างเป็น .ipynb	ทุกไฟล์

10 สรุปทั้งระบบ + คำศัพท์

เล่าทั้งระบบใน 6 ประโยค

1. เรามีสามเหลี่ยมและอยากขยับมุมมันด้วย gradient descent แต่การเรนเดอร์ปกติ “หาอนุพันธ์ไม่ได้” เพราะขอบเป็นฟังก์ชันขั้นบันได
2. แก้ด้วยการทำขอบให้นุ่ม: $\alpha = \text{sigmoid}(\text{ระยะถึงขอบ} / \sigma)$ — ตอนนี้อยู่บนเส้นเรียบและหาอนุพันธ์ได้
3. ภาพจริงต้องคม จึง “สุ่ม” $\text{Bernoulli}(\alpha)$ ได้ 0/1 คมๆ แต่ค่าคาดหวังยังเท่า α ที่เรียบ
4. Forward: ยิงรังสี 1 ล้านเส้น (1 thread/เส้น) หาจุดตัดด้วย Möller–Trumbore $\rightarrow \alpha \rightarrow$ สุ่ม
5. Backward: chain rule $\partial L / \partial V = (\partial L / \partial \alpha) (\partial \alpha / \partial d) (\partial d / \partial V)$ สะสมด้วย atomicAdd กัน race
6. ทุก Step มีตัวทดสอบเทียบ CPU/finite-difference ก่อนไปต่อ — สร้างแบบ incremental และตรวจสอบได้

คำศัพท์ (Glossary)

คำ	ความหมายสั้นๆ
Differentiable rendering	การเรนเดอร์ที่หาอนุพันธ์เทียบพารามิเตอร์จากได้ $\Rightarrow$ เทรนด้วย gradient descent ได้
Barycentric (w,u,v)	พิกัดถ่วงน้ำหนัก 3 มุม บอกตำแหน่งในสามเหลี่ยม; ตัวใด=0 คืออยู่บนขอบ
Möller–Trumbore	อัลกอริทึมหาจุดตัดรังสี-สามเหลี่ยม คำนวณ (t,u,v) ในครั้งเดียว
σ (sigma)	ความกว้างแถบนุ่มรอบขอบ; เล็ก=ขอบคม, gradient กระจุก (โจทย์ใช้ 0.01)
$\text{Bernoulli}(\alpha)$	การสุ่มออกหัว/ก้อย ที่มีโอกาสออก “1” = α
Monte Carlo	ประมาณค่าด้วยการสุ่มเฉลี่ยหลายครั้ง; คลาดเคลื่อน $\sim 1/\sqrt{N}$
REINFORCE / score-function	วิธีหาอนุพันธ์ของค่าคาดหวังของกระบวนการสุ่ม
thread / block / grid	ลำดับชั้นการทำงานขนานบน CUDA
Philox	RNG แบบ counter-based เหมาะกับ GPU; (seed,index) $\rightarrow$ เลขสุ่ม
atomicAdd	บวกค่าลงหน่วยความจำแบบ “อ่าน-บวก-เขียน” ไม่มีใครแทรก กัน race
Race condition	หลาย thread เขียนที่เดียวกันพร้อมกันจนข้อมูลหาย (lost update)
FP ordering jitter	ผลต่างระดับ bit จากลำดับบวก floating point — ไม่ใช่ bug
Finite differences	ประมาณอนุพันธ์ด้วย $(f(x+\epsilon) - f(x-\epsilon)) / 2\epsilon$ ใช้ตรวจ gradient analytic

แหล่งอ้างอิง

- DiffSoup project page: kenji-tojo.github.io/publications/diffsoup/
- GitHub (โค้ดทางการ): github.com/kenji-tojo/diffsoup
- งานวิจัย: Tojo, Bickel, Umetani — CVPR 2026

เช็คลิสต์ความเข้าใจ (ลองตอบตัวเอง)

1) ทำไม binary opacity หาคณพันธ์ไม่ได้? 2) การสุมช่วยใหหาคณพันธ์ได้อย่างไร? 3) ทำไม gradient ไหลผ่าน α ไม่ใช่ผ่าน sample? 4) atomicAdd แกั race อย่างไร และทำไม FP jitter ไม่ใช่ race? ถ้าตอบได้ทั้ง 4 ข้อ = เข้าใจหัวใจของโปรเจกต์นี้แล้ว